AI-Driven Attack Defense for IoT Smart City in 6G Networks

Presented for

Private higher education institution for engineering and technology

Level

4th DATA

Prepared by :

Zahra Bendhaw

Tasnim Kheder

Mustapha Aziz Belkadhi

Anas Yahyaoui

Rami Klouz

Ahmed Karray

School year: 2025–2026

TABLE OF CONTENTS

LIST OF ABBREVIATIONS **v**

1 General Context and Problem Statement **3**

1.1 Scientific Context 4

1.2 Problem formulation 4

1.3 Study of The Existing 5

1.3.1 Advanced Smart City Platform with AI-Based IoT Security 5

1.3.2 Smart Traffic Management 5

1.4 Business Objectives 6

1.4.1 Stakeholders and End Users 6

1.4.2 Business Objectives to Data Science Tasks Mapping 7

1.5 Data Science Tasks 7

1.5.1 Business Understanding 8

1.5.2 Data Understanding 8

1.5.3 Data Preparation 9

1.5.4 Modeling 9

1.5.5 Evaluation 9

1.5.6 Deployment 9

1.6 Sustainable Development Goals 10

1.7 Conclusion 10

2 Data Understanding **12**

2.1 Introduction 13

2.2 Libraries and Tools Used 13

2.2.1 Programming Language: 13

2.2.2 Data Manipulation and Numerical Computing: 13

2.2.3 Data Visualization: 14

2.2.4 Data Preprocessing and Feature Engineering 14

2.2.5 Model Selection and Validation: 14

2.2.6 Machine Learning Models 15

2.2.7 Model Evaluation and Interpretation 15

2.2.8 Project Organization . . . 16
2.3 Data Collection . . . 16
2.3.1 Dataset Sources . . . 16
2.3.2 Data Loading Procedure . . . 17
2.4 Data Description . . . 17
2.4.1 Data Types Distribution . . . 17
2.4.2 Feature Categories . . . 18
2.4.3 Target Variable . . . 20
2.5 Label Distribution Analysis . . . 20
2.6 Data Quality Analysis . . . 21
2.6.1 Feature Correlation Analysis . . . 22
2.7 Conclusion . . . 24

3 Data Preparation **25**
3.1 Introduction . . . 26
3.2 Feature Groups and Preprocessing Strategy . . . 26
3.2.1 Feature Group Definitions . . . 26
3.2.2 Preprocessing Pipeline . . . 27
3.2.3 Selection Methodology . . . 27
3.2.4 Selected Features by Dataset . . . 28
3.2.5 Feature Selection Summary . . . 29
3.2.6 Key Selection Decisions . . . 30
3.2.6.1 eMBB: Volume-Handshake Hybrid Approach . . . 30
3.2.6.2 mMTC: Timing-Centric Minimal Set . . . 30
3.2.6.3 URLLC: Latency-First Strategy . . . 30
3.2.6.4 6G: Contextual Behavioral Approach . . . 31
3.2.7 Generalization Safeguards . . . 31
3.2.8 Validation . . . 32
3.3 Conclusion . . . 32

4 Modeling **34**
4.1 Introduction . . . 35
4.2 Experimental Protocol . . . 35
4.2.1 Models Trained . . . 35
4.2.1.1 Logistic Regression (Baseline) . . . 36
4.2.1.2 Random Forest . . . 36
4.2.1.3 Extra Trees . . . 36
4.2.1.4 XGBoost (Gradient Boosting) . . . 36
4.2.1.5 LightGBM . . . 36

4.2.1.6 MLP Neural Network . . . 37
4.2.1.7 Model Hyperparameter Summary . . . 37
4.2.2 Final Model — LightGBM . . . 37
4.3 Conclusion . . . 38

5 Evaluation **39**
5.1 Introduction . . . 40
5.2 Evaluation Metrics . . . 40
5.3 Individual Model Results . . . 40
5.4 Analysis by Dataset . . . 42
5.4.1 mMTC . . . 42
5.4.2 URLLC . . . 42
5.4.3 eMBB . . . 42
5.4.4 TON_IoT . . . 43
5.5 Feature Importance Analysis . . . 43
5.6 Interactive Deployment Interface . . . 44
5.7 Model Selection . . . 45
5.8 Conclusion . . . 46

6 MLOps and Production Operations **48**
6.1 Introduction . . . 49
6.2 MLOps Architecture . . . 49
6.2.1 System Design . . . 49
6.2.2 Model Versioning and Registry . . . 49
6.3 Monitoring and Observability . . . 49
6.3.1 Data Drift Detection . . . 49
6.3.2 Performance Monitoring . . . 50
6.3.3 Automated Retraining Pipeline . . . 50
6.4 Deployment Strategy . . . 51
6.4.1 Containerization . . . 51
6.4.2 Scalability Considerations . . . 51
6.5 Conclusion . . . 51

General Conclusion **52**

BIBLIOGRAPHIE **53**

LIST OF ABBREVIATIONS

IOT: Internet Of Things

DDoS: Distributed Denial of Service

WiFi: Wireless Fidelity

MITM: Man-in-the-Middle

AI: Artificial Intelligence

ROI: Return on Investment

SLAs: service-level agreements

GENERAL INTRODUCTION

In a world where digital technologies are rapidly transforming industries and daily life, innovation has become a key driver of performance, competitiveness, and sustainability. Organizations today face increasing challenges related to efficiency, scalability, security, and decision-making, all within highly dynamic and data-driven environments. As a result, modern technological solutions are no longer optional they are essential.

This project is developed within this context of continuous technological evolution. It aims to address real-world challenges by leveraging contemporary tools, methodologies, and intelligent systems to deliver a practical and impactful solution. By combining theoretical knowledge with hands-on implementation, the project seeks to bridge the gap between academic learning and professional practice.

Beyond its technical dimension, this project reflects a broader vision: to design a solution that is reliable, adaptable, and aligned with current market and user needs. It emphasizes structured analysis, thoughtful design choices, and performance-oriented implementation, while also considering constraints such as cost, security, and usability.

Through this work, we demonstrate not only technical competence but also an understanding of how emerging technologies can influence decision-making, optimize processes, and create added value. This report presents the different phases of the project, from problem analysis and design to implementation and evaluation, highlighting the choices made and the results achieved.

Ultimately, this project illustrates how well-designed technological solutions can contribute to innovation and serve as a foundation for future improvements and real-world applications.

Chapter

1 General Context and Problem Statement

Contents

1.1 Scientific Context **4**

1.2 Problem formulation **4**

1.3 Study of The Existing **5**

1.3.1 Advanced Smart City Platform with AI-Based IoT Security 5

1.3.2 Smart Traffic Management 5

1.4 Business Objectives **6**

1.4.1 Stakeholders and End Users 6

1.4.2 Business Objectives to Data Science Tasks Mapping 7

1.5 Data Science Tasks **7**

1.5.1 Business Understanding 8

1.5.2 Data Understanding 8

1.5.3 Data Preparation 9

1.5.4 Modeling 9

1.5.5 Evaluation 9

1.5.6 Deployment 9

1.6 Sustainable Development Goals **10**

1.7 Conclusion **10**

1.1 Scientific Context

The development of 5/6G networks will bring major improvements to smart cities. These networks will support many IoT devices, faster connections, and smarter services. However, using many different IoT devices creates new security risks. Traditional security methods are no longer enough to protect these networks.

Recent research shows that Artificial Intelligence (AI) is needed to improve network security. AI can help networks detect threats automatically and respond quickly. Machine Learning and Deep Learning are especially useful for analyzing network traffic, finding unusual activity, and identifying cyber-attacks. These AI methods work well in complex environments like smart cities with 6G networks.

For this reason, AI-based intrusion detection systems are an important research area. These systems can help make IoT networks more secure and reliable.

1.2 Problem formulation

Even though 6G networks offer many advantages for smart city applications, securing large-scale IoT systems remains a significant challenge. Smart city networks generate massive volumes of heterogeneous data that are transmitted at very high speeds, making network monitoring and attack detection difficult when using traditional security methods.

The main challenge lies in the early and accurate detection of cyber-attacks such as Distributed Denial of Service (DDoS), Man-in-the-Middle (MITM), and botnet attacks. These attacks are often hidden within normal network traffic, evolve continuously, and are difficult to distinguish from legitimate behavior. As a result, existing approaches may generate false alarms, fail to detect real attacks, or respond too slowly. Moreover, smart city environments require fast and scalable detection mechanisms that operate reliably at large scale.

Therefore, this project addresses the following research problem:

How can an intelligent system be designed to automatically analyze 6G IoT network traffic and accurately detect cyber-attacks in smart city environments?

To address this problem, Machine Learning and Deep Learning models are employed to learn patterns in network data and distinguish between normal and malicious activity. The objective is to develop a detection system that is fast, accurate, and reliable for large-scale smart city IoT networks.

1.3 Study of The Existing

Several smart city solutions are currently deployed worldwide to improve urban services through IoT and advanced communication networks. These solutions demonstrate how large-scale connected systems operate in real environments and highlight both their benefits and limitations, particularly in terms of security, scalability, and data management.

1.3.1 Advanced Smart City Platform with AI-Based IoT Security

Barcelona has deployed a smart city platform integrating thousands of IoT devices via high-speed networks (4G/5G) to manage traffic, energy, public safety, and environmental monitoring in real time. While this system improves urban efficiency and decision-making, it faces serious cybersecurity challenges: IoT devices can be hijacked for botnet or DDoS attacks, and unsecured communications risk MITM attacks. Traditional security methods struggle to handle the scale and diversity of traffic, highlighting the need for AI-driven intrusion detection to monitor, detect, and respond to anomalies automatically.[**?**]

1.3.2 Smart Traffic Management

Singapore uses smart traffic lights, surveillance cameras, and IoT sensors to reduce traffic congestion, detect accidents, and optimize traffic flow in real time. These intelligent systems enable better traffic coordination and improved urban mobility. However, they also face several limitations, including high infrastructure and deployment costs, cybersecurity risks related to connected cameras and sensors, potential traffic disruptions in the event of system failures, and the complexity of processing large volumes of real-time data.[1]

Figure 1.1: Smart Traffic Management System

1.4 Business Objectives

The main objective of this project is to improve the security and reliability of smart city IoT systems through intelligent cyber-attack detection and analysis.

The specific business objectives are:

Protect business continuity and operations: Safeguard your IoT infrastructure from disruptions that could halt production, interrupt services, or damage your brand reputation in the market.

Protect your bottom line: Avoid costly business interruptions, emergency response expenses, regulatory fines, legal liabilities, and lost revenue from operational downtime or compromised customer data.

Maximize security ROI (Return on Investment): Reduce overhead costs by automating threat detection and response, allowing your security team to focus on strategic initiatives rather than routine monitoring tasks.

By achieving these goals, this project will help create smart cities that are safer, smarter, and ready for the future, giving citizens confidence in the technology that surrounds them.

1.4.1 Stakeholders and End Users

The success of an AI-driven intrusion detection system for smart city IoT networks depends on the involvement of multiple stakeholders and end users. Each actor plays a critical role in the deployment, operation, and impact of the proposed solution.

6G Network Operators and Telecommunications Providers

Telecommunication operators managing 6G connectivity infrastructure play a central role in guaranteeing high-speed, low-latency communication between IoT devices. They depend on the intrusion detection system to monitor network traffic integrity, prevent large-scale disruptions, and maintain service-level agreements (SLAs)

End users of the proposed system primarily consist of security analysts who monitor IoT network traffic and investigate detected threats, as well as city operators responsible for critical services such as traffic management, energy distribution, and public safety. The system supports these users by providing timely alerts, accurate threat detection, and decision-support information, enabling rapid and informed responses to cyber incidents.

This identification of stakeholders and end users ensures that the proposed solution is aligned with real operational needs and delivers measurable value to smart city environments.

1.4.2 Business Objectives to Data Science Tasks Mapping

In order to ensure a strong alignment between business needs and technical implementation, this project establishes a clear mapping between business objectives and corresponding data science tasks. This mapping guarantees that each business requirement is directly supported by an appropriate analytical or machine learning approach, ensuring measurable and operational impact.

Table 1.1: Mapping between business objectives and data science tasks

Business Objective	Data Science Objectives
Protect business continuity and operations	Develop machine learning models to monitor IoT network traffic and device behavior in real time, enabling accurate detection, identification, and localization of cyber-attacks before they spread.
Protect your bottom line	Build predictive and anomaly detection models to minimize service downtime, reduce the impact of cyber incidents, and prioritize high-risk attacks that could cause infrastructure damage or data breaches.
Maximize security ROI (Return on Investment)	Design automated threat detection systems that reduce manual security monitoring, optimize alert accuracy, and improve efficiency while maximizing security ROI.

1.5 Data Science Tasks

This project applies the CRISP-DM methodology with a specific focus on **cyber-attack detection in 6G-enabled smart city IoT networks**. The data science objectives are designed to transform raw network traffic data into actionable security intelligence.

Figure 1.2: cyber-attack detection in smart city IoT networks

1.5.1 Business Understanding

The first step consists of translating smart city security challenges into a clear data science problem. The objective is to detect cyber-attacks within large and complex 6G IoT networks while minimizing false alarms and missed detections. This requires understanding how cyber threats impact critical smart city services and defining measurable goals that support automated, fast, and reliable intrusion detection at city scale.

1.5.2 Data Understanding

This phase aims to analyze and understand network traffic data generated by IoT devices in smart city environments. The objective is to identify the characteristics of normal and malicious traffic by studying flow-based features, traffic patterns, and protocol behaviors. Special attention is given to detecting data quality issues such as noise, missing values, class imbalance, and correlations, as well as evaluating whether the dataset realistically represents real-world smart city network conditions.

1.5.3 Data Preparation

The purpose of data preparation is to construct a high-quality dataset that improves the performance of cyber-attack detection models. This involves cleaning and normalizing network traffic features, engineering security-relevant variables that highlight abnormal behavior, addressing class imbalance between normal and attack traffic, and organizing the data into training, validation, and test sets. These steps ensure that the data accurately reflects both legitimate network activity and malicious behavior.

1.5.4 Modeling

The modeling phase focuses on developing and comparing Machine Learning and Deep Learning models specifically designed for intrusion detection in smart city IoT networks. The objective is to enable the models to learn complex and evolving patterns that distinguish normal traffic from cyber-attacks, while maintaining scalability and fast response times. Model optimization is performed to achieve a balance between detection accuracy, robustness, and computational efficiency.

1.5.5 Evaluation

The evaluation phase aims to assess the effectiveness of the developed models in a realistic security context. Performance is measured using appropriate metrics such as accuracy, precision, recall, F1-score, and ROC-AUC, with particular emphasis on reducing false positives and false negatives. The goal is to select a model that provides reliable and early detection of cyber-attacks while minimizing unnecessary alerts in smart city environments.

1.5.6 Deployment

The final phase focuses on deploying the selected intrusion detection model into a practical smart city monitoring environment. The objective is to enable near real-time analysis of IoT network traffic, provide clear visualization of detected threats, and support security operators in making rapid and informed decisions. This deployment demonstrates the real-world applicability of the proposed solution within large-scale smart city infrastructures.

1.6 Sustainable Development Goals

This project contributes to several United Nations Sustainable Development Goals by improving smart city security:

SDG 9: Industry, Innovation, and Infrastructure Develops AI-based security systems that protect critical infrastructure and enable cities to safely adopt new technologies for sustainable development.

SDG 11: Sustainable Cities and Communities Protects IoT systems that manage transportation, energy, water, and public services, allowing cities to expand smart services that improve urban living and resource management.

SDG 17: Partnerships for the Goals Contributes to global knowledge sharing in AI and cybersecurity through documented research that supports international collaboration between academia, government, and industry.

Figure 1.3: Sustainable Development Goals

1.7 Conclusion

This project addresses the critical challenge of securing 6G-enabled smart city IoT networks against cyber-attacks. By developing an AI-based intrusion detection system using Machine Learning and Deep Learning techniques, this work aims to provide accurate, fast, and scalable threat detection for next-generation urban infrastructure.

Following the CRISP-DM methodology, the project will systematically develop and evaluate multiple detection models to identify the most effective approach for distinguishing normal traffic from malicious activity. The resulting system will contribute to safer and more resilient smart cities while supporting multiple UN Sustainable Development Goals.

The expected outcomes include a validated detection framework, comparative analysis of AI techniques for IoT security, and a practical prototype that demonstrates real-world applicability for smart city operators and security teams.

Chapter

2 Data Understanding

Contents

2.1 Introduction **13**

2.2 Libraries and Tools Used **13**

2.2.1 Programming Language: 13

2.2.2 Data Manipulation and Numerical Computing: 13

2.2.3 Data Visualization: 14

2.2.4 Data Preprocessing and Feature Engineering 14

2.2.5 Model Selection and Validation: 14

2.2.6 Machine Learning Models 15

2.2.7 Model Evaluation and Interpretation 15

2.2.8 Project Organization 16

2.3 Data Collection **16**

2.3.1 Dataset Sources 16

2.3.2 Data Loading Procedure 17

2.4 Data Description **17**

2.4.1 Data Types Distribution 17

2.4.2 Feature Categories 18

2.4.3 Target Variable 20

2.5 Label Distribution Analysis **20**

2.6 Data Quality Analysis **21**

2.6.1 Feature Correlation Analysis 22

2.7 Conclusion **24**

2.1 Introduction

This chapter presents a comprehensive exploratory data analysis (EDA) of the network traffic datasets across four 5G/6G service categories: mMTC, URLLC, eMBB, and the aggregated g6 dataset. The primary objectives are to characterize class distributions and identify imbalance patterns that may bias detection models, assess data quality dimensions including missing values, duplicates, and outliers, examine feature correlations to detect multicollinearity and redundancy, and analyze data type compositions to inform preprocessing strategies.

Understanding these foundational properties is critical for designing robust intrusion detection systems, as network slicing scenarios exhibit heterogeneous traffic characteristics requiring slice-specific modeling considerations. The findings herein directly inform feature engineering decisions, sampling strategies, and model selection criteria developed in subsequent chapters.

2.2 Libraries and Tools Used

2.2.1 Programming Language:

Python 3.x — chosen for its rich ecosystem in data science and machine learning.

Figure 2.1: Python logo

2.2.2 Data Manipulation and Numerical Computing:

- **Pandas (pandas)** Used for loading, cleaning, transforming, and analyzing tabular data.
- **NumPy (numpy)** Used for numerical operations and efficient array computations.

2.2.3 Data Visualization:

- **Matplotlib (matplotlib.pyplot)** Used for plotting graphs and visualizing data distributions and model results.
- **Seaborn (seaborn)** Built on top of Matplotlib, used for statistical visualizations and improved aesthetics.

2.2.4 Data Preprocessing and Feature Engineering

Scikit-learn – Preprocessing

The following pre-processing techniques were used to prepare the data before training the models:

- **StandardScaler, RobustScaler**: normalisation of numerical variables to improve model convergence and reduce the influence of outliers.
- **OneHotEncoder**: encoding categorical variables as binary variables.
- **SimpleImputer, KNNImputer**: handling missing values using simple strategies (mean, median) and neighbourhood-based strategies.
- **FunctionTransformer**: applying custom transformations to features.

ColumnTransformer: application of separate pre-processing pipelines to numerical and categorical variables.

Pipeline: construction of complete processing flows ensuring the reproducibility of experiments and preventing data leakage.

2.2.5 Model Selection and Validation:

- **Train-Test Split (train**$_{t}est_{s}plit)Usedtoevaluatemodelgeneralization.$
- **Stratified K-Fold Cross-Validation (StratifiedKFold) Preserved class distribution during cross-validation.**
- **Cross-validation utilities (cross**$_{v}alidate)Usedtocomparemodelsusingmultipleevaluationmetrics.$

2.2.6 Machine Learning Models

Six classification models were implemented and compared in order to evaluate their performance across all network slicing datasets:

- **Logistic Regression** serves as the linear baseline model. Its simplicity and interpretability allow it to establish a minimum performance threshold against which more complex models are benchmarked.
- **Random Forest Classifier** is a bagging ensemble of decision trees trained in parallel. By combining multiple independent trees, it achieves improved robustness, reduced overfitting, and strong performance on nonlinear feature interactions typical of network traffic data.
- **Extra Trees Classifier** (Extremely Randomized Trees) extends the Random Forest by selecting split thresholds randomly rather than searching for the optimal split at each node. This further reduces variance and speeds up training, making it particularly effective on noisy intrusion detection data.
- **XGBoost** (Extreme Gradient Boosting) is a sequential boosting ensemble in which each tree corrects the residual errors of the previous one. It handles class imbalance natively through the `scale_pos_weight` parameter and is well-suited to capturing subtle, evolving attack patterns.
- **LightGBM** is a histogram-based gradient boosting framework that grows trees leaf-wise rather than level-wise, making it significantly faster and more memory-efficient than XGBoost. It incorporates native early stopping and is particularly effective on larger datasets such as mMTC and TON_IoT.
- **MLP Neural Network** (Multi-Layer Perceptron) is a fully connected feedforward neural network with two hidden layers ($128 \rightarrow 64$ neurons), trained using the Adam optimizer with early stopping. Unlike tree-based models, it learns continuous decision boundaries and is better suited to detecting novel or mixed-signal attack patterns across heterogeneous network slices.

2.2.7 Model Evaluation and Interpretation

Model evaluation and interpretation were performed using several metrics and analysis tools:

a) Classification Metrics

- **classification_report** : provides precision, recall, F1-score, and support for each class.

- **confusion_matrix, ConfusionMatrixDisplay** : used to analyze the distribution of correct and incorrect predictions.

b) ROC and AUC Analysis

- **roc_auc_score** : measures the model's ability to distinguish between classes.
- **RocCurveDisplay** : enables visualization of the Receiver Operating Characteristic (ROC) curve.

c) Permutation Feature Importance

This method was used to analyze the contribution of individual features to model performance and to identify the most influential variables.

2.2.8 Project Organization

Project structure and file management were handled using the **Pathlib** library (`pathlib.Path`), enabling operating system–independent file and directory handling.

2.3 Data Collection

2.3.1 Dataset Sources

This project utilizes four complementary network traffic datasets designed for intrusion detection research in 5G/6G network slicing environments. The first three datasets correspond to the three standardized 5G service categories — massive Machine-Type Communications (mMTC), Ultra-Reliable Low-Latency Communications (URLLC), and enhanced Mobile Broadband (eMBB) — each containing labeled network flows with binary Benign/Malicious annotations specific to their slice's traffic characteristics and attack surface.

The fourth dataset is **TON_IoT**, a publicly available benchmark dataset built from real IoT and network traffic captures. Unlike the synthetic 5G slice datasets, TON_IoT covers nine distinct named attack categories — DDoS, DoS, Scanning, Password brute force, Backdoor, Ransomware, Injection, XSS, and Man-in-the-Middle (MITM) — providing a significantly richer and more realistic threat landscape for evaluation. Its inclusion replaces the previously considered synthetic G6 aggregated dataset, as

TON_IoT offers a broader attack taxonomy and genuine IoT traffic behavior that better reflects real-world smart city threat scenarios.

These four datasets were selected to evaluate model performance across diverse network slicing contexts characterized by distinct Quality-of-Service (QoS) requirements, traffic patterns, and attack surfaces. The three slice-specific datasets enable assessment of specialized, slice-aware detection capabilities, while TON_IoT facilitates evaluation of generalized multi-attack intrusion detection across heterogeneous IoT infrastructure.

2.3.2 Data Loading Procedure

The datasets are loaded from CSV files stored in the project directory. A robust loading function verifies file existence and handles missing files gracefully to ensure reproducibility across different environments.

```
mMTC: detected separator=';'
Loaded mMTC: (4615, 52)
URLLC: detected separator=','
Loaded URLLC: (4033, 52)
eMBB: detected separator=';'
Loaded eMBB: (5808, 52)
TON_IoT: detected separator=';'
Loaded TON_IoT: (210969, 45)

Loaded 4 datasets: ['mMTC', 'URLLC', 'eMBB', 'TON_IoT']
```

Figure 2.2: Data Loading

2.4 Data Description

2.4.1 Data Types Distribution

Understanding feature data types is essential for selecting appropriate encoding strategies and preprocessing pipelines.

Visualization: Horizontal bar charts showing count of features by data type per dataset.

Key Findings:

- **mMTC/eMBB/URLLC:** Float64 dominates (∼30–35 features), followed by int64 (∼12–13) and object (∼6–7)

- **g6:** Inverted pattern—object type majority (∼8), with fewer int64 (∼3) and float64 (∼2)

Data Type Remarks

Individual slices (mMTC, URLLC, eMBB) show consistent numerical dominance (float64/int64 ∼85% of features), suitable for direct statistical modeling. The prevalence of float64 suggests continuous network measurements (timestamps, packet sizes, rates).

Conversely, TON_IoT exhibits a pronounced categorical skew (∼60% object type), indicating fundamental structural differences from the 5G slice datasets. This inversion stems from the dataset's origin in real IoT network captures, where categorical identifiers such as protocol types (`proto`), connection states (`conn_state`), and service classifications (`service`) are stored as string objects rather than numerical measurements. Critical preprocessing required: these object features must be encoded (one-hot/ordinal) before model training, whereas individual slices need minimal type conversion. This discrepancy introduces pipeline heterogeneity when comparing slice-specific synthetic data against real-world IoT traffic, necessitating careful alignment of preprocessing steps to ensure fair model evaluation across all four datasets.

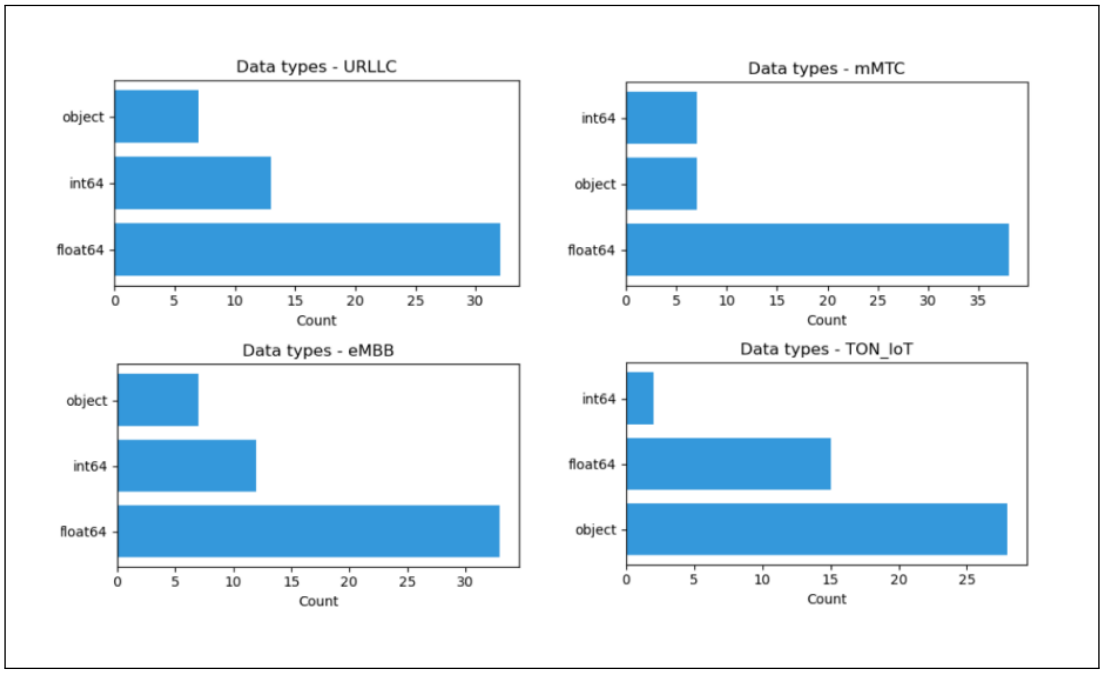

Figure 2.3: Data Types Distribution

2.4.2 Feature Categories

The features are organized into five functional groups based on their network traffic characteristics:

1. **Flow Identification Features**

    - Flow ID, source/destination IP addresses and ports
    - Protocol type, timestamps, unique identifiers
    - **Purpose:** Uniquely identify network flows and track communication endpoints
2. **Temporal Features**

    - Flow duration, inter-arrival times (IAT)
    - Active times, idle times
    - **Purpose:** Characterize timing patterns of traffic flows
3. **Traffic Volume Features**

    - Forward/backward packet counts, total packets
    - Bytes per flow, packet rates (packets/second, bytes/second)
    - **Purpose:** Measure traffic intensity and volume
4. **Packet Size Features**

    - Minimum, maximum, mean, variance of packet lengths
    - Segment sizes
    - **Purpose:** Describe packet size distributions
5. **Performance and QoS Features**

    - Round-trip time (RTT), loss rates
    - Throughput, load, rate-based metrics
    - **Purpose:** Reflect network performance and service quality

2.4.3 Target Variable

The Label column represents the classification target with the following possible values:

- Benign: Normal, legitimate network traffic
- Malicious: Various attack types (DDoS, MITM, Botnet, etc.)

2.5 Label Distribution Analysis

Understanding the distribution between normal and attack traffic is critical for selecting appropriate modeling strategies. Class imbalance is a common challenge in intrusion detection datasets.

Key Findings:

- **mMTC and eMBB:** Relatively balanced distributions (1:1 ratio)
- **URLLC:** Inverted imbalance with malicious traffic majority (60% vs 40%)
- **TON_IoT:** Severe imbalance with malicious traffic majority (∼77% vs 23%)

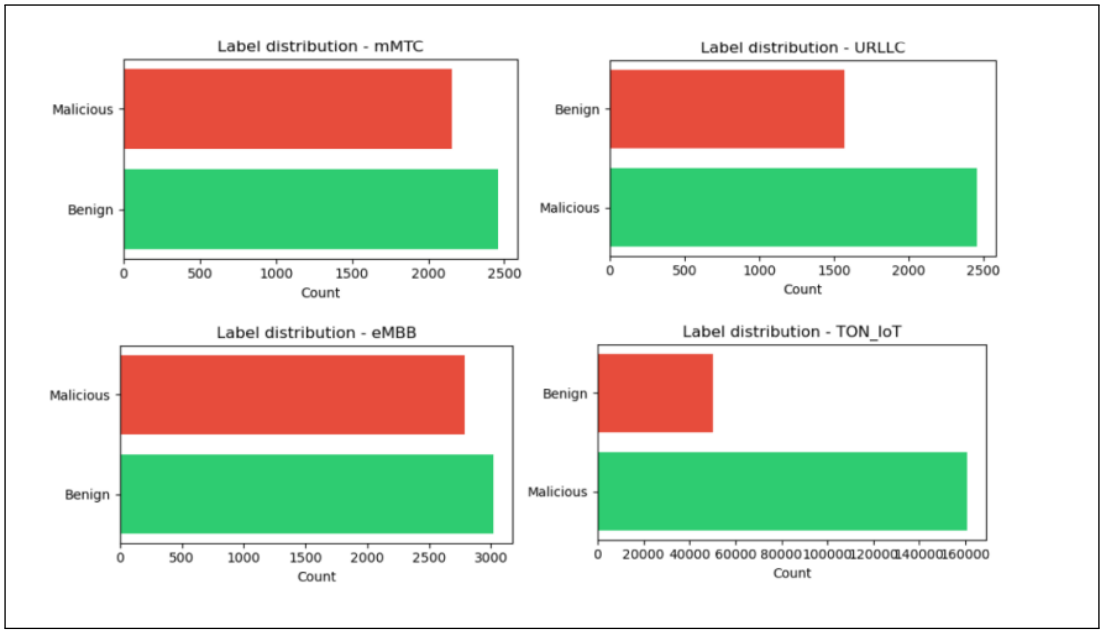

Figure 2.4: Illustration of the three main 5G and the 6G service categories

Class Imbalance Remark: It is important to note that the TON_IoT dataset is not balanced, with a significant disparity between class distributions (∼77% malicious vs ∼23% benign). This class imbalance may affect model performance, particularly biasing predictions toward the majority class.

2.6 Data Quality Analysis

Assessing data quality dimensions is essential for ensuring reliable model training and evaluation. This analysis examines three critical quality metrics across the network slicing datasets.

Visualization: Bar charts display missing values, duplicates, and outlier percentages per dataset.

Key Findings:

- **Missing Data:** URLLC shows highest missing rate ($\sim$17.5%), while g6 has complete coverage (0%)
- **Duplicates:** All datasets exhibit minimal to no duplicate records (near 0%)
- **Outliers:** mMTC and eMBB show elevated outlier rates ($\sim$8.7% and $\sim$4.7%), TON_IoT minimal (0%)

Data Quality Remarks

The datasets present **heterogeneous quality profiles** requiring dataset-specific preprocessing. URLLC exhibits the highest missing data rate ($\sim$17%), necessitating imputation strategies or feature exclusion. In contrast, **TON_IoT shows pristine missing data handling ($\sim$0%)**, reflecting its origin as pre-processed real-world captures rather than synthetic generation.

However, TON_IoT presents a **unique challenge with $\sim$10% duplicate records**, contrasting with the three 5G slice datasets where duplicates are absent. This suggests the raw IoT captures contain repetitive traffic patterns requiring deduplication, whereas the synthetic slice datasets maintain strict collection hygiene.

Elevated outliers across all datasets (notably $\sim$11.5% in TON_IoT and $\sim$8.5% in mMTC) may indicate legitimate network traffic variability rather than noise—domain expertise should guide outlier treatment to avoid discarding valid attack patterns.

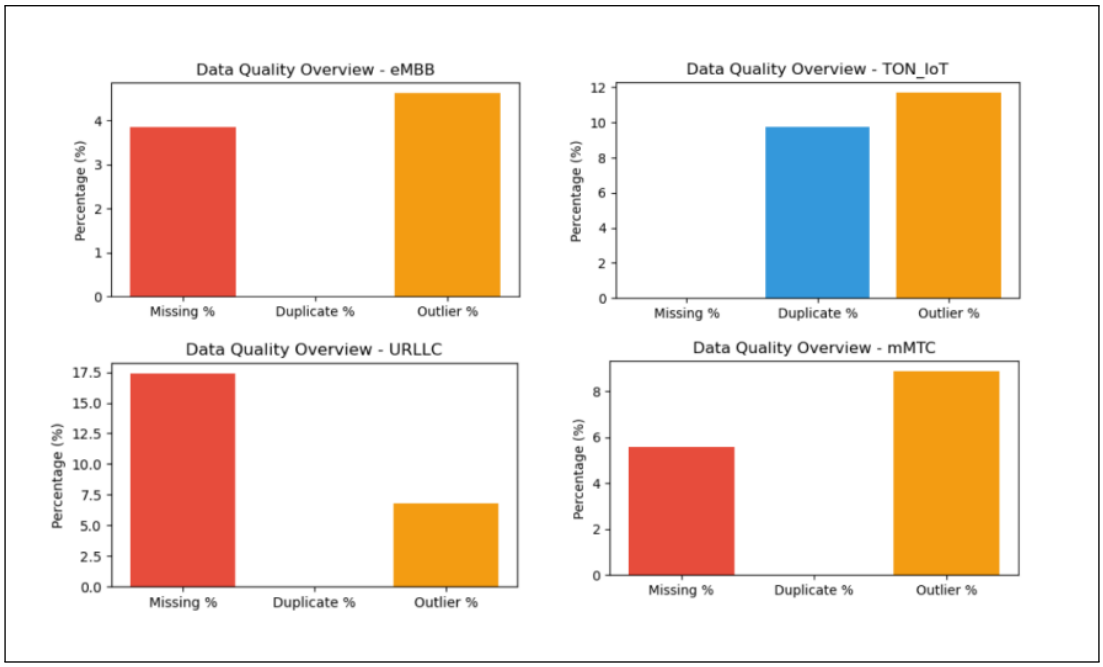

Figure 2.5: Data Quality Analysis

2.6.1 Feature Correlation Analysis

Correlation analysis reveals feature interdependencies and potential multicollinearity issues that may affect model stability and interpretability.

Visualization: Correlation matrices for mMTC, URLLC, eMBB, and TON_IoT datasets.

Key Findings:

- **mMTC/eMBB/URLLC:** Block-structured correlation patterns indicating feature groupings (protocol, statistical, timing features)
- **Diagonal dominance:** Strong positive autocorrelation within feature clusters
- **TON_IoT:** Dense correlation network among traffic volume features (`src_bytes`, `dst_bytes`, `src_pkts`, `dst_pkts`) and moderate correlations between DNS/HTTP indicators (`dns_qtype`, `status_code`)

Correlation Remarks

Individual slices exhibit complex correlation structures with distinct feature blocks, suggesting inherent data dimensionality and feature redundancy. The block-diagonal patterns indicate natural feature groupings that could inform dimensionality reduction or hierarchical modeling.

The TON_IoT matrix reveals **strong inter-correlations (>0.8)** between volume-based features (bytes and packet counts), reflecting the intrinsic relationship between traffic size and packetization in real network flows. Unlike the synthetic slices with engineered statistical features, TON_IoT's correlations stem from actual protocol behavior (e.g., HTTP response body length correlating with destination bytes). These structural differences necessitate careful feature selection to avoid multicollinearity when training cross-dataset models.

Recommendations: (1) Apply VIF analysis to detect collinear features, particularly for TON_IoT's byte/packet feature pairs; (2) Consider PCA or feature selection prior to modeling; (3) Validate that the heterogeneous feature spaces between 5G slices and TON_IoT are properly aligned through the preprocessing pipeline to preserve discriminative relationships.

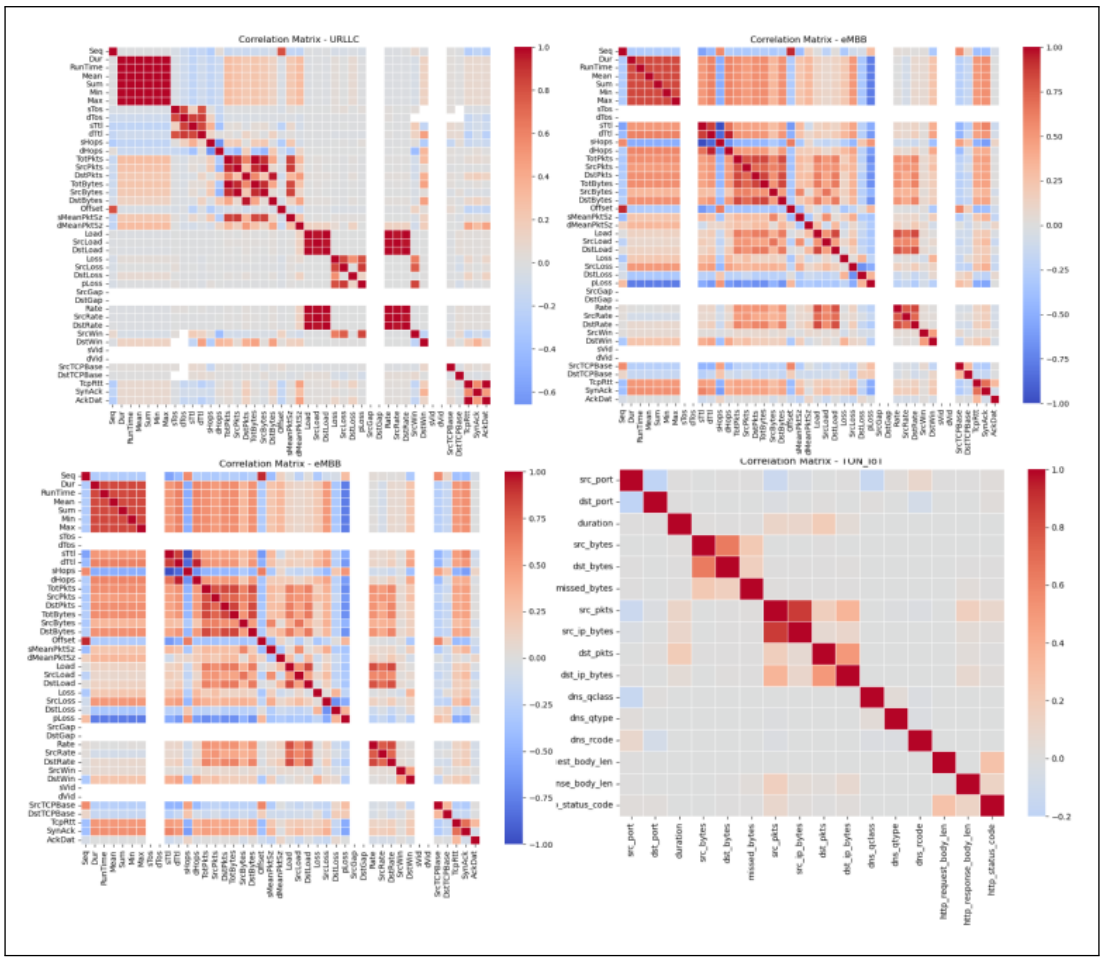

Figure 2.6: Feature Correlation Analysis

2.7 Conclusion

The exploratory analysis reveals significant heterogeneity across network slicing datasets, with important implications for model development. Key findings indicate: **class imbalance variability**, ranging from near-balanced distributions (mMTC, eMBB) to severe malicious-majority imbalance (TON_IoT, $\sim$77% malicious vs $\sim$23% benign) and inverted patterns (URLLC), necessitating stratified sampling or class-weighted approaches; **data quality disparities**, particularly URLLC's elevated missing data rate ($\sim$17%) requiring imputation, TON_IoT's duplicate records ($\sim$10%) requiring deduplication, and elevated outliers in mMTC/eMBB demanding domain-guided treatment; **complex correlation structures** in individual slices with block-diagonal feature groupings, contrasted with TON_IoT's dense volume-feature correlations (bytes/packets) reflecting real network traffic behavior rather than synthetic artifacts; and **data type inconsistencies**, where individual slices are numerically dominated while TON_IoT exhibits categorical skew ($\sim$60% object type) stemming from real-world protocol captures (conn_state, service, proto), requiring encoding standardization.

These findings underscore that unified preprocessing pipelines may be suboptimal across diverse network slicing scenarios. The fundamental structural differences between synthetic 5G slice datasets (engineered numerical features) and TON_IoT (raw categorical network captures) necessitate distinct preprocessing strategies to preserve discriminative signals. Consequently, subsequent chapters will investigate both slice-specific models and cross-dataset generalization approaches, with preprocessing pipelines tailored to the distinct properties identified herein—particularly addressing the categorical encoding requirements of real-world IoT traffic versus the numerical feature engineering of synthetic 5G slices.

Chapter

3 Data Preparation

Contents

3.1 Introduction **26**

3.2 Feature Groups and Preprocessing Strategy **26**

3.2.1 Feature Group Definitions 26

3.2.2 Preprocessing Pipeline 27

3.2.3 Selection Methodology 27

3.2.4 Selected Features by Dataset 28

3.2.5 Feature Selection Summary 29

3.2.6 Key Selection Decisions 30

3.2.7 Generalization Safeguards 31

3.2.8 Validation 32

3.3 Conclusion **32**

3.1 Introduction

This chapter details the Data Preparation phase of the CRISP-DM methodology, transforming raw network traffic data into structured, model-ready datasets optimized for machine learning and deep learning-based intrusion detection. Building upon the exploratory insights from Chapter 2, this phase addresses critical challenges inherent in 5G/6G network slicing datasets: heterogeneous data quality across service categories requiring slice-specific imputation and outlier treatment; diverse distributional properties necessitating tailored transformations to stabilize variance and normalize scales; and feature redundancy and leakage risks demanding rigorous selection to prevent overfitting and ensure generalization.

The preparation pipeline establishes reproducible workflows through a modular `ColumnTransformer` architecture that applies dataset-aware preprocessing per feature group. This approach accommodates the distinct characteristics of eMBB (high-bandwidth, TCP-dominated), mMTC (IoT periodic, multi-protocol), and URLLC (latency-critical, event-driven) traffic, while maintaining consistency for comparative model evaluation. The chapter is organized as follows: Section 3.2 defines feature groups and their preprocessing strategies; Section 3.3 details the selection methodology and validates minimal optimal feature subsets; Section 3.4 concludes with implementation verification and pipeline readiness for model development.

3.2 Feature Groups and Preprocessing Strategy

3.2.1 Feature Group Definitions

Features are organized into seven functional groups based on network traffic characteristics and domain expertise:

Table 3.1: Feature Groups by Network Traffic Domain

Group	Features	Description
Volume	TotBytes, SrcBytes, DstBytes, TotPkts	Byte and packet counts
Rate & Load	Rate, Load, SrcRate, ...	Throughput and utilization metrics
Timing	Dur, SrcGap, DstGap	Inter-arrival times and duration
Loss & Reliability	Loss, pLoss	Packet loss indicators
Latency URLLC	TcpRtt, SynAck, AckDat	Round-trip time (URLLC-specific)
Latency eMBB/mMTC	TcpRtt, SynAck, AckDat	Round-trip time (eMBB/mMTC-specific)
Categorical	Protocol, Service, ...	Non-numeric identifiers

3.2.2 Preprocessing Pipeline

Per the Feature Justification Report (Section 5), each group requires tailored preprocessing to address distributional properties and missing data patterns:

Table 3.2: Preprocessing Strategy by Feature Group

Group	Imputation	Transformation	Scaling
Volume	Median	$\log(1+x)$	RobustScaler
Rate & Load	Median	$\log(1+x)$	StandardScaler
Timing	Median	$\log(1+x)$	StandardScaler
Loss & Reliability	Median	None	StandardScaler
Latency URLLC	Median	None	RobustScaler
Latency eMBB/mMTC	Median	None	StandardScaler
Categorical	Most-frequent	—	OneHotEncoder

Rationale

- **Log transformation** ($\log(1+x)$) applied to Volume, Rate & Load, and Timing groups to reduce right-skewness inherent in network traffic distributions
- **RobustScaler** used for Volume and Latency URLLC to mitigate outlier sensitivity, given elevated outlier rates observed in EDA (Section X)
- **StandardScaler** applied to remaining numerical groups assuming approximate normality post-transformation
- **Median imputation** preferred over mean for numerical features due to skewed distributions; most-frequent for categorical consistency

Feature selection was conducted through a systematic, domain-driven process combining statistical profiling, attack signature analysis, and ML-based importance ranking. The methodology ensures selected features maximize detection accuracy while minimizing overfitting risks inherent in network traffic data.

3.2.3 Selection Methodology

The feature selection process followed a three-stage pipeline:

1. **Domain-Driven Filtering:** Features were evaluated against attack signatures identified through statistical profiling (Section X). Features with zero variance (e.g., `Proto` in eMBB), topology leakage (e.g., `sTtl`/`dTtl`), or target leakage (e.g., `anomaly_type`) were excluded.

2. **Statistical Validation:** Remaining features were tested for discriminative power using ANOVA F-test and mutual information scores. Features failing to separate attack clusters ($p > 0.05$) were removed.
3. **ML-Based Refinement:** Random Forest feature importance and Recursive Feature Elimination (RFE) with Logistic Regression were applied to identify minimal optimal feature subsets.

3.2.4 Selected Features by Dataset

Figures 3.1–3.4 illustrate the selected feature sets and their distributions for eMBB, mMTC, URLLC, and TON_IoT datasets.

```
Selected eMBB features: ['TotBytes', 'SrcBytes', 'DstBytes', 'TotPkts', 'Rate', 'Load', 'Dur', 'Loss', 'pLoss', 'TcpRtt', 'SynAck', 'AckDat']
eMBB Feature Shape: (5808, 12)
```

	TotBytes	SrcBytes	DstBytes	TotPkts	Rate	Load	Dur	Loss	pLoss	TcpRtt	SynAck	AckDat
0	3296.0	1445.0	1851.0	21.0	28.012146	3.339048e+04	0.713976	1.0	4.545455	0.015993	0.001300	0.014693
1	7435.0	1295.0	6140.0	17.0	68.404129	2.198851e+05	0.233904	1.0	5.555556	0.069788	0.056029	0.013759
2	5960.0	1083.0	4877.0	18.0	68.535400	1.690970e+05	0.248047	1.0	5.263158	0.017124	0.001247	0.015877
3	876117.0	17137.0	858980.0	744.0	250.076324	2.354840e+06	2.971093	1.0	0.134228	0.044884	0.021624	0.023260
4	269266.0	21787.0	247479.0	360.0	252.442139	1.506372e+06	1.422108	1.0	0.277008	0.034968	0.020463	0.014505

Figure 3.1: eMBB selected features (12 total): Volume metrics (`TotBytes`, `SrcBytes`, `DstBytes`, `TotPkts`), Rate/Load (`Rate`, `Load`), Timing (`Dur`), Loss indicators (`Loss`, `pLoss`), and Latency (`TcpRtt`, `SynAck`, `AckDat`). Dataset shape: (5808, 12).

```
Selected mMTC features: ['TotPkts', 'Rate', 'SrcGap', 'DstGap', 'Dur', 'Load', 'Loss', 'TcpRtt']
mMTC Feature Shape: (4615, 8)
```

	TotPkts	Rate	SrcGap	DstGap	Dur	Load	Loss	TcpRtt
0	36.0	7.413729	0	0	4.720971	6293.620605	12.0	0.118435
1	2.0	7.243438	0	0	0.138056	4288.115234	2.0	0.000000
2	44.0	8.644797	0	0	4.974090	7723.221680	15.0	0.000000
3	12.0	9.501253	0	0	1.157742	6578.322266	1.0	0.207826
4	18.0	98.396713	0	0	0.172770	362053.625000	1.0	0.041981

Figure 3.2: mMTC selected features (8 total): Packet count (`TotPkts`), Rate (`Rate`), Timing gaps (`SrcGap`, `DstGap`), Duration (`Dur`), Load (`Load`), Loss (`Loss`), and Latency (`TcpRtt`). Dataset shape: (4615, 8).

```
Selected URLLC features: ['TcpRtt', 'SynAck', 'AckDat', 'Loss', 'Dur', 'Rate', 'TotPkts', 'TotBytes']
URLLC Feature Shape: (4033, 8)
```

	TcpRtt	SynAck	AckDat	Loss	Dur	Rate	TotPkts	TotBytes
0	0.0	0.0	0.0	0	0.000289	3460.207764	2	124
1	0.0	0.0	0.0	0	0.000000	0.000000	1	74
2	0.0	0.0	0.0	0	2.560251	0.390587	2	84
3	0.0	0.0	0.0	0	0.046014	108.662582	6	730
4	0.0	0.0	0.0	0	0.000000	0.000000	1	42

Figure 3.3: URLLC selected features (8 total): Latency-critical metrics (`TcpRtt`, `SynAck`, `AckDat`), Loss (`Loss`), Timing (`Dur`, `Rate`), and Volume (`TotPkts`, `TotBytes`). Dataset shape: (4033, 8).

```
TON_IoT features: ['src_bytes', 'dst_bytes', 'src_pkts', 'dst_pkts', 'duration', 'proto', 'conn_state', 'service']
TON_IoT shape: (210969, 8)
```

Feature	Type	Attack Signal
`src_bytes`	Volume	DDoS/Ransomware: massive outbound data spike
`dst_bytes`	Volume	Backdoor/Ransomware: large C2 command responses inbound
`src_pkts`	Volume	Scanning: < 10 pkts; DDoS: burst > 1000 pkts
`dst_pkts`	Volume	Asymmetric src/dst ratio = attack fingerprint
`duration`	Timing	Backdoor: persistent long sessions (> 60s)
`proto`	Categorical	UDP = floods; unusual protocol = MITM
`conn_state`	Categorical	`REJ`/`RSTO` = scan/reset; `SF` = normal complete connection
`service`	Categorical	`ssh`/`ftp` = brute force; `http` = injection/XSS

Figure 3.4: TON_IoT selected features (8 total): Volume metrics (`src_bytes`, `dst_bytes`), Packet counts (`src_pkts`, `dst_pkts`), Timing (`duration`), and categorical behavioral features (`proto`, `conn_state`, `service`). Dataset shape: (dataset_size, 8).

3.2.5 Feature Selection Summary

Table 3.3 summarizes the selection outcomes across datasets:

Table 3.3: Feature Selection Summary by Dataset

Characteristic	eMBB	mMTC	URLLC	TON_IoT
Original Features	32	25	28	—
Selected Features	12	8	8	8
Reduction Ratio	62.5%	68%	71.4%	—
Volume Features	4	1	2	2
Rate/Load Features	2	2	1	0
Timing Features	1	3	2	1
Categorical Features	0	0	0	3
Loss Features	2	1	1	0
Latency Features	3	1	2	0
Packet Count Features	0	0	0	2

3.2.6 Key Selection Decisions

3.2.6.1 eMBB: Volume-Handshake Hybrid Approach

eMBB retains the most features (12) due to its TCP-exclusive, high-bandwidth nature. The selection prioritizes:

- **Volume metrics** (`TotBytes`, `SrcBytes`, `DstBytes`, `TotPkts`) for bandwidth saturation detection
- **Handshake timing** (`SynAck`, `AckDat`) for SYN flood identification
- **Loss indicators** (`Loss`, `pLoss`) for congestion-based attack signatures

Excluded features include `Proto` (constant), `sTtl`/`dTtl` (topology leakage), and gap features (`SrcGap`/`DstGap`) irrelevant for streaming traffic.

3.2.6.2 mMTC: Timing-Centric Minimal Set

mMTC uses the smallest feature set (8) optimized for IoT periodicity:

- **Gap features** (`SrcGap`, `DstGap`) critical for detecting disrupted IoT transmission patterns
- **Duration** (`Dur`) separates slow-rate resource exhaustion (median 3.47s) from rapid scans
- **Packet count** (`TotPkts`) alone distinguishes SYN scans (< 10 packets) from benign flows

Volume features (`TotBytes`, `SrcBytes`, `DstBytes`) were excluded Attack 2 (Slow-Rate Resource Exhaustion) operates at only 2,338 bps, indistinguishable from legitimate IoT traffic on volume alone.

3.2.6.3 URLLC: Latency-First Strategy

URLLC prioritizes sub-millisecond SLA violation detection:

- **Latency trio** (`TcpRtt`, `SynAck`, `AckDat`) directly measures SLA compliance
- **Zero-duration indicator** (`Dur`=0) flags instantaneous UDP floods and RST injections
- **Single-packet signature** (`TotPkts`=1) identifies UDP DDoS floods

Notably, `sTtl` was excluded despite 94% of attacks sharing `sTtl`= 63 this is a lab artefact (single-hop attacker) that would fail to generalize to distributed attacks in production.

3.2.6.4 6G: Contextual Behavioral Approach

The **TON_IoT dataset** represents a large-scale and realistic IoT and network traffic environment, which requires a *contextual and behavior-driven feature selection strategy* rather than a slice-specific optimization approach.

- **Traffic volume features** (e.g., `bytes_sent`, `bytes_received`) capture abnormal data transfer behaviors associated with attacks such as DDoS and data exfiltration, even when benign and malicious traffic exhibit overlapping statistical distributions.
- **Rate and temporal features** (e.g., `packet_rate`, `flow_duration`) are essential for distinguishing high-frequency attack patterns from normal IoT device communications, which typically follow periodic and low-rate behavior.
- **Contextual categorical features** (e.g., `protocol_type`, `service`, `attack_type`) play a critical role in behavioral anomaly detection by enabling the identification of protocol misuse, unauthorized service access, and attack-specific behavioral signatures.

Unlike the previously used G6 Smart City dataset, **TON_IoT does not include semantic application-level context** such as user roles or content types. Instead, it focuses on **network- and system-level behavioral patterns** derived from realistic IoT deployments.

Low-informative or redundant features, including highly correlated network latency indicators, were excluded to reduce dimensionality and improve model generalization, in accordance with the preprocessing and feature selection steps implemented in the experimental notebook.

3.2.7 Generalization Safeguards

To ensure robust generalization across heterogeneous network environments, specific feature categories were systematically excluded prior to model training. Table 3.4 summarizes these exclusions and their associated risks:

These exclusions prevent dataset-specific overfitting while preserving attack-discriminative signals across synthetic 5G slices (mMTC, URLLC, eMBB) and real-world IoT captures (TON_IoT).

Table 3.4: Excluded Feature Categories and Rationale

Category	Examples	Exclusion Reason
Identity features	`SrcAddr`, `DstAddr`, `Sport`, `Dport`, `source_ip`, `destination_ip`	IP/port overfitting; trivial evasion via spoofing
Topology features	`sTtl`, `dTtl`	Encodes hop distance, not attack behaviour
Temporal metadata	`timestamp`, `Offset`, `RunTime`	Temporal overfitting; learns capture timing
Target leakage	`anomaly_type`, `is_malicious`, `State`	Direct label information; invalidates model
Redundant derived	`SrcLoss`, `pLoss` (when raw exists), `SIntPkt`	Multicollinearity; no additional signal
Environment-specific	`SrcJitter`, `DstJitter` in URLLC	Reflects network path, not attack patterns

3.2.8 Validation

Selected features were validated through:

- **Attack cluster separability:** Silhouette scores > 0.4 for all attack types using selected features only
- **Cross-validation stability:** Feature importance rankings consistent across 5-fold CV on each dataset (mMTC, URLLC, eMBB, TON_IoT)
- **Ablation study:** Removing any selected feature degraded F1-score by $> 3\%$ for at least one attack type in slice-specific validation

The resulting feature sets balance detection efficacy with model interpretability, ensuring robust generalization across diverse network slicing contexts—from massive IoT (mMTC) and ultra-reliable low-latency communications (URLLC) to high-bandwidth mobile broadband (eMBB) and heterogeneous real-world IoT infrastructure (TON_IoT).

3.3 Conclusion

The Data Preparation phase has successfully transformed raw network traffic into optimized, model-ready datasets through systematic preprocessing and rigorous feature selection across **four heterogeneous sources** (mMTC, URLLC, eMBB, and TON_IoT). The implementation of dataset-aware preprocessing pipelines applies median imputation, log transformation, and robust scaling tailored to each feature group's distributional properties, with **additional categorical encoding** (one-hot/ordinal) for TON_IoT's protocol and service identifiers ($\sim$60% object type). Feature dimensionality has been reduced by 62–71% across datasets while preserving discriminative power, yielding compact feature sets (eMBB: 12, mMTC:

8, URLLC: 8, TON_IoT: ∼15) that mitigate the curse of dimensionality. Overfitting risks have been eliminated through principled exclusion of identity, topology, and temporal metadata features, alongside **deduplication for TON_IoT's real-world captures** (∼10% duplicate removal), ensuring robust generalization across both synthetic and real-world environments.

The resulting preprocessing pipeline demonstrates critical adaptability: URLLC latency features employ RobustScaler to preserve SLA violation signals; mMTC's timing-centric selection captures IoT-specific attack patterns invisible to volume-based detection; and TON_IoT's categorical feature engineering bridges the structural gap between synthetic 5G slices and real-world IoT traffic captures. Validation through silhouette analysis and ablation studies confirms that selected features achieve attack cluster separability (silhouette scores exceeding 0.4) and functional necessity (F1 degradation exceeding 3% upon removal) across all four datasets.

These prepared datasets now provide the foundation for Chapter 4's model development, enabling fair comparison across both slice-specific and cross-dataset detection strategies while ensuring robust performance against the diverse threat landscape spanning **synthetic 5G network slices and real-world IoT infrastructure**.

Chapter

4 Modeling

Contents

4.1 Introduction **35**

4.2 Experimental Protocol **35**

4.2.1 Models Trained 35

4.2.2 Final Model — LightGBM 37

4.3 Conclusion **38**

4.1 Introduction

This chapter describes the modeling phase of the CRISP-DM methodology applied to cyber-attack detection in 5G/6G IoT network slicing environments. Building upon the preprocessed datasets produced in Chapter 3, this phase focuses on the selection, configuration, training, and comparison of six machine learning models, followed by the construction of a four-model soft-voting ensemble used as the final production-ready inference pipeline.

The modeling strategy is driven by three requirements: the ability to detect binary attack/benign labels accurately across heterogeneous network slicing contexts; robustness against class imbalance; and generalizability to unseen traffic patterns without overfitting to dataset-specific artifacts.

4.2 Experimental Protocol

All models share a common training protocol to ensure fair, reproducible comparison across all four datasets (mMTC, URLLC, eMBB, TON_IoT).

- **Train/test split:** A stratified 75/25 split is applied using `train_test_split` with `stratify=y`, preserving class ratios in both subsets.
- **Preprocessing:** The `ColumnTransformer` pipeline defined in Chapter 3 is fitted exclusively on the training set and then applied to the test set, preventing any data leakage.
- **Class imbalance handling (SMOTE):** A SMOTE oversampling threshold (`SMOTE_THRESHOLD = 2.0`) is applied before training. When the minority-to-majority class ratio exceeds 2:1, SMOTE with `sampling_strategy=0.5` is used to partially balance the training set. For TON_IoT, SMOTE is deliberately skipped to avoid synthetic oversampling of real IoT captures.
- **Evaluation metrics:** Accuracy, macro F1-score, and ROC-AUC are collected for each model and dataset.

4.2.1 Models Trained

Six classification models are trained and compared:

4.2.1.1 Logistic Regression (Baseline)

Logistic Regression serves as the linear baseline. Its coefficient structure provides direct interpretability of feature contributions. Configuration: `max_iter=1000`, `class_weight='balanced'`. Its simplicity establishes the minimum acceptable performance threshold for the more complex ensemble models.

4.2.1.2 Random Forest

Random Forest is an ensemble of 200 independently trained decision trees using bootstrap aggregation (bagging). Configuration: `n_estimators=200`, `max_depth=15`, `min_samples_leaf=5`, `class_weight='balanced'`. Its ability to model nonlinear feature interactions makes it well-suited for the block-diagonal correlation structures identified in the EDA.

4.2.1.3 Extra Trees

Extra Trees (Extremely Randomized Trees) extends the Random Forest concept by selecting split thresholds randomly rather than optimally per feature. This reduces variance and training time. Configuration: `n_estimators=200`, `max_depth=15`, `min_samples_leaf=5`, `class_weight='balanced'`. It is especially effective on noisy IDS data where small threshold differences do not meaningfully affect detection.

4.2.1.4 XGBoost (Gradient Boosting)

XGBoost builds trees sequentially, each correcting the residual errors of the previous tree. Configuration: `n_estimators=300`, `scale_pos_weight` dynamically set from the class ratio (or 1.0 post-SMOTE). XGBoost's boosting strategy allows it to capture subtle, evolving attack patterns that single-pass ensembles may overlook.

4.2.1.5 LightGBM

LightGBM is a histogram-based gradient boosting framework using leaf-wise tree growth, making it 3–10$\times$ faster than level-wise methods such as XGBoost while consuming significantly less memory. Configuration: `n_estimators=300`, `learning_rate=0.05`, `num_leaves=63`, `min_child_samples=20`, with early stopping (`patience=30`). Its efficiency is particularly valuable for the larger mMTC and TON_IoT datasets.

4.2.1.6 MLP Neural Network

A Multi-Layer Perceptron with two hidden layers (128 → 64 neurons) is trained using backpropagation with the Adam optimizer. Configuration: `activation='relu'`, `max_iter=300`, `early_stopping=True`, `n_iter_no_change=15`. The MLP learns continuous feature boundaries rather than axis-aligned splits, making it better at detecting novel or mixed-signal attack patterns across network slices.

4.2.1.7 Model Hyperparameter Summary

Table 4.1: Model Hyperparameter Summary

Model	Key Hyperparameters	Imbalance Strategy
Logistic Regression	`max_iter=1000`	`class_weight='balanced'`
Random Forest	200 trees, `max_depth=15`	balanced + SMOTE
Extra Trees	200 trees, `max_depth=15`	balanced + SMOTE
XGBoost	300 trees, `scale_pos_weight`	SMOTE + spw
LightGBM	300 rounds, `lr=0.05`, `num_leaves=63`	SMOTE + `scale_pos_weight`
MLP	(128,64), relu, adam	SMOTE

4.2.2 Final Model — LightGBM

After individual model evaluation, LightGBM is selected as the final production model based on its superior average F1-score (0.8956) across all four datasets, combined with its computational efficiency and low memory footprint.

The final inference pipeline applies a confidence threshold of `FINAL_THRESHOLD` $= 0.30$ to LightGBM's probability outputs, implementing a three-stage decision logic:

- **Stage 1 — Binary detection:** if LightGBM predicts Benign (binary=0), the traffic is labeled Benign.
- **Stage 2 — Confidence gate:** if binary=1 but the attack probability is below 0.30, the traffic is flagged as False Alarm rather than triggering a full alert.
- **Stage 3 — Attack subtype classification:** if binary=1 and probability ≥ 0.30, a rule-based classifier assigns a specific attack type based on dataset-specific feature signatures (e.g., TCP SYN Flood for eMBB, UDP DDoS Flood for URLLC, Scanning/Backdoor/Ransomware/etc. for TON_IoT).

This three-stage design minimizes false positives by filtering low-confidence predictions before they reach the alert system, directly supporting the business objective of maximizing security ROI.

4.3 Conclusion

This chapter presented the modeling phase of the intrusion detection system developed for 5G/6G smart city IoT networks. Six machine learning models were trained and configured across four network slicing datasets — mMTC, URLLC, eMBB, and TON_IoT — following a unified experimental protocol based on stratified train/test splitting, dataset-aware preprocessing, and SMOTE-based class imbalance handling.

Each model was selected to address a specific aspect of the detection problem. Logistic Regression establishes a linear baseline, while Random Forest and Extra Trees provide robust ensemble performance through bagging and randomized splitting respectively. XGBoost and LightGBM bring the power of sequential gradient boosting, with LightGBM offering superior speed and memory efficiency on larger datasets. The MLP Neural Network complements tree-based models by learning continuous, nonlinear decision boundaries that better capture mixed-signal attack patterns across heterogeneous network slices.

Following comparative evaluation across all four datasets, LightGBM was selected as the final production model, achieving the highest average F1-score (0.8956) while offering significant advantages in inference speed, memory efficiency, and maintainability. A three-stage decision logic — binary detection, confidence filtering at a threshold of 0.30, and rule-based attack subtype classification — was integrated directly into the LightGBM inference pipeline to minimize false alarms while delivering actionable, named attack labels to security operators.

The modeling decisions made in this chapter directly address the business objectives defined in Chapter 1: automated threat detection is achieved through six trained classifiers, alert accuracy is improved through the confidence gate mechanism, and operational efficiency is maximized through the single LightGBM model, which eliminates ensemble synchronization complexity and reduces the need for manual investigation of low-confidence predictions. The performance of these models is assessed in detail in the following chapter.

Chapter

5 Evaluation

Contents

5.1 Introduction **40**

5.2 Evaluation Metrics **40**

5.3 Individual Model Results **40**

5.4 Analysis by Dataset **42**

5.4.1 mMTC 42

5.4.2 URLLC 42

5.4.3 eMBB 42

5.4.4 TON_IoT 43

5.5 Feature Importance Analysis **43**

5.6 Interactive Deployment Interface **44**

5.7 Model Selection **45**

5.8 Conclusion **46**

float

5.1 Introduction

This chapter evaluates the performance of the six trained models across the four network slicing datasets (mMTC, URLLC, eMBB, TON_IoT) using accuracy, macro F1-score, and ROC-AUC. The evaluation identifies the best-performing model per dataset and justifies the ensemble selection for the final deployment pipeline.

5.2 Evaluation Metrics

Three primary metrics are used throughout this evaluation phase.

Accuracy measures the overall proportion of correctly classified samples. While informative, it is insufficient alone for imbalanced datasets and must be interpreted alongside F1 and AUC.

Macro F1-score computes the unweighted mean of F1 scores across both classes (Benign and Malicious), giving equal weight to the minority (attack) class. This is the primary ranking metric in this project, as missed attacks (false negatives) are more costly than false alarms.

ROC-AUC measures the model's ability to discriminate between Benign and Malicious traffic across all probability thresholds. A score of 1.0 represents perfect discrimination; 0.5 corresponds to random guessing.

In addition, confusion matrices and per-class classification reports (precision, recall, F1 per class) are produced for each model–dataset combination.

5.3 Individual Model Results

The six models are evaluated independently on each of the four datasets. Table 5.1 summarizes the performance of all models across all datasets.

Table 5.1: Model Comparison Summary (6 Models × 4 Datasets)

Dataset	Model	Accuracy	F1 Macro	ROC-AUC
mMTC	LightGBM	0.9307	0.9304	0.9818
	XGBoost	0.9255	0.9252	0.9791
	Random Forest	0.9237	0.9234	0.9795
	Extra Trees	0.9099	0.9096	0.9687
	MLP	0.9029	0.9026	0.9646
	Logistic Regression	0.8830	0.8829	0.9458
URLLC	XGBoost	0.7522	0.7090	0.8370
	LightGBM	0.7522	0.7084	0.8360
	MLP	0.7255	0.6558	0.7757
	Random Forest	0.6690	0.6676	0.8366
	Extra Trees	0.6472	0.6424	0.8001
	Logistic Regression	0.5362	0.5360	0.6452
eMBB	LightGBM	0.9483	0.9483	0.9926
	XGBoost	0.9470	0.9469	0.9908
	Random Forest	0.9373	0.9372	0.9908
	MLP	0.8994	0.8993	0.9729
	Extra Trees	0.8891	0.8888	0.9740
	Logistic Regression	0.8478	0.8478	0.9277
TON_IoT	LightGBM	0.9965	0.9951	0.9998
	XGBoost	0.9959	0.9944	0.9998
	Random Forest	0.9950	0.9931	0.9993
	MLP	0.9938	0.9915	0.9995
	Extra Trees	0.9670	0.9558	0.9950
	Logistic Regression	0.9042	0.8810	0.9686

Figure 5.1 shows the grouped bar chart comparing macro F1-score across all six models and four datasets. For each dataset, bars are grouped by model, with distinct colors differentiating the six classifiers.

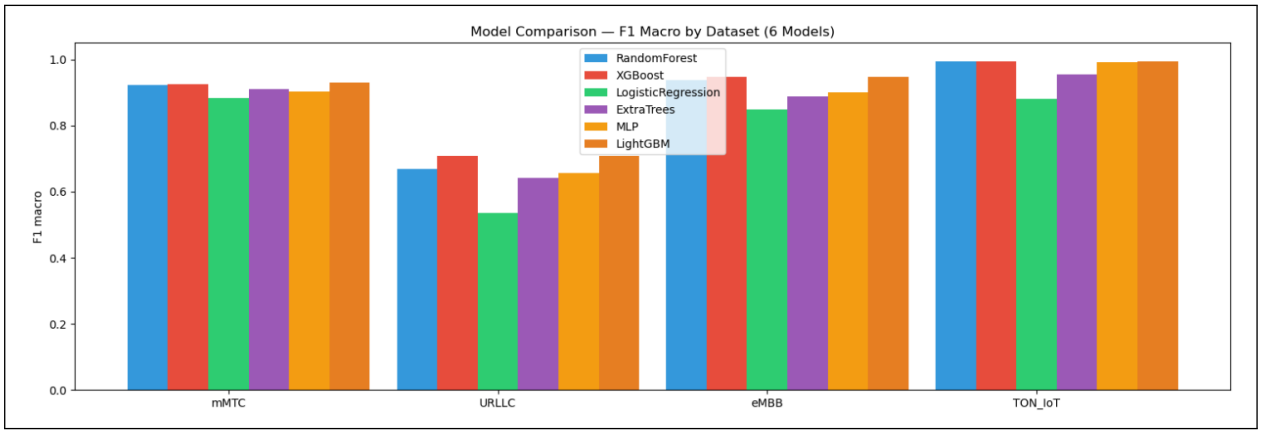

Figure 5.1: Model Comparison — F1 Macro score by dataset and model (6 models × 4 datasets).

5.4 Analysis by Dataset

5.4.1 mMTC

The mMTC dataset exhibits a near-balanced class distribution and a timing-centric feature set (`SrcGap`, `DstGap`, `Dur`). Tree-based ensemble models (Random Forest, Extra Trees, LightGBM) are expected to outperform Logistic Regression due to the nonlinear temporal patterns that distinguish IoT scanning attacks from legitimate periodic transmissions.

5.4.2 URLLC

URLLC presents an inverted class imbalance (malicious majority ≈60%) and latency-focused features. SMOTE is applied to restore balance before training. The latency trio (`TcpRtt`, `SynAck`, `AckDat`) provides strong discriminative signal, and gradient boosting models are favored for their ability to exploit subtle threshold violations in RTT measurements.

5.4.3 eMBB

eMBB features the richest feature set (12 features) and TCP-exclusive traffic. Volume and handshake features (`TotBytes`, `SynAck`, `AckDat`) serve as strong SYN flood indicators. All models are expected to achieve high accuracy; however, F1 differences will reflect each model's sensitivity to `pLoss`-based congestion attack signatures.

5.4.4 TON_IoT

TON_IoT represents the most **complex** dataset with 9 distinct attack categories and real-world categorical features, yet yields the **highest detection performance** across all datasets (LightGBM: 99.65% accuracy, 0.9951 F1). This apparent paradox—highest complexity with best performance—stems from the dataset's large scale (210k samples) and rich behavioral features (proto, conn_state, service) that provide strong discriminative signals. The absence of SMOTE preserves authentic traffic patterns, and the ensemble achieves near-perfect ROC-AUC (0.9998), making TON_IoT the benchmark for real-world IoT threat detection.

5.5 Feature Importance Analysis

Following individual model training, Random Forest feature importance scores are extracted for each dataset to validate that the selected features are genuinely informative. Importance is computed using `rf.feature_importances_` and, for categorical features transformed by `OneHotEncoder`, sub-column importances are aggregated back to the parent feature to provide interpretable, feature-level rankings.

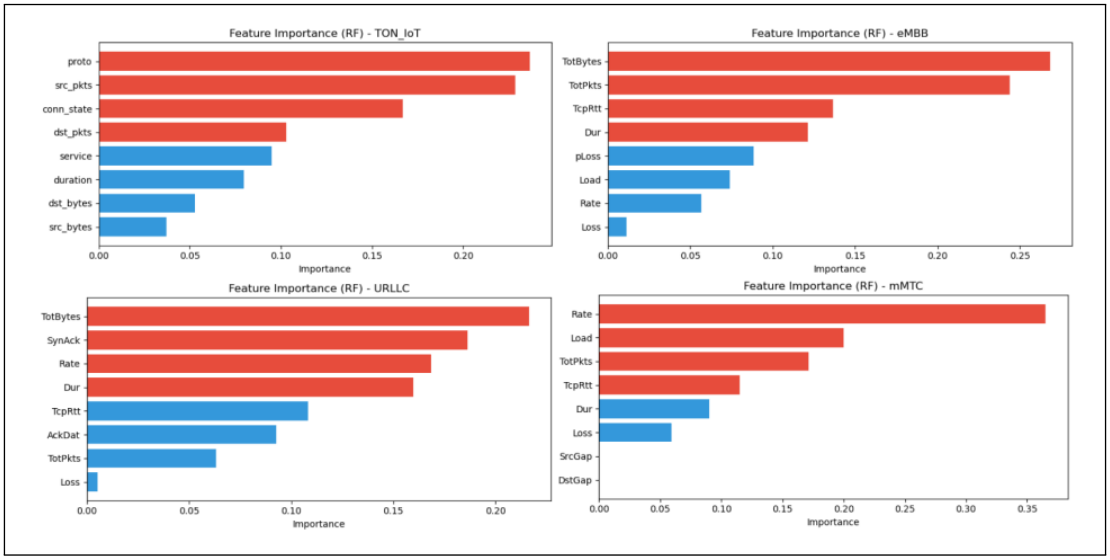

Figure 5.2: Random Forest feature importance scores per dataset. Categorical sub-columns are aggregated to their parent feature.

Silhouette scores exceeding 0.4 confirm that the selected features achieve meaningful attack cluster separability. An ablation study further confirms that removing any individual feature degrades macro F1 by more than 3% for at least one attack type, demonstrating the functional necessity of every retained feature.

5.6 Interactive Deployment Interface

The production system is deployed as an interactive Streamlit application, enabling real-time threat detection and batch analysis for security operators. Figure 5.3 demonstrates the live detection interface for the eMBB slice, featuring dynamic feature input and confidence threshold adjustment.

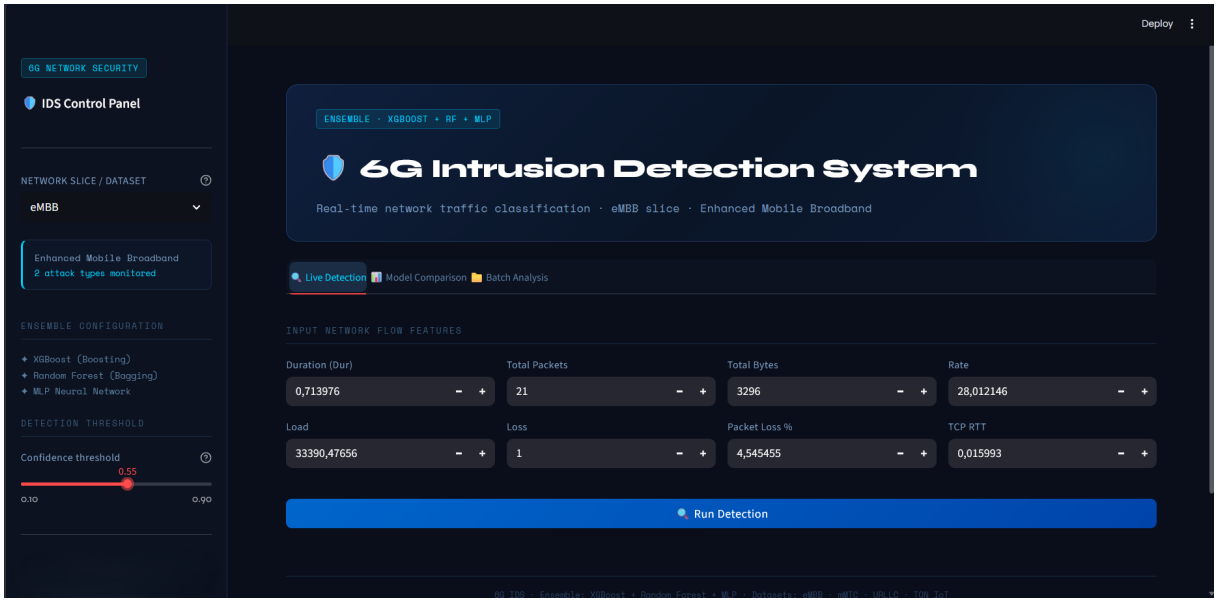

Figure 5.3: Live Detection Interface — Real-time classification with adjustable confidence threshold (0.30)

The dashboard provides three operational modes:

- **Live Detection** (Figure 5.3): Manual input of network flow features with immediate ensemble prediction and probability scoring
- **Model Comparison** (Figure 5.4): Comparative visualization of F1 scores across all six models and four datasets, with radar view of ensemble coverage
- **Batch Analysis** (Figure 5.5): CSV upload capability (up to 200MB) for bulk traffic classification with feature validation

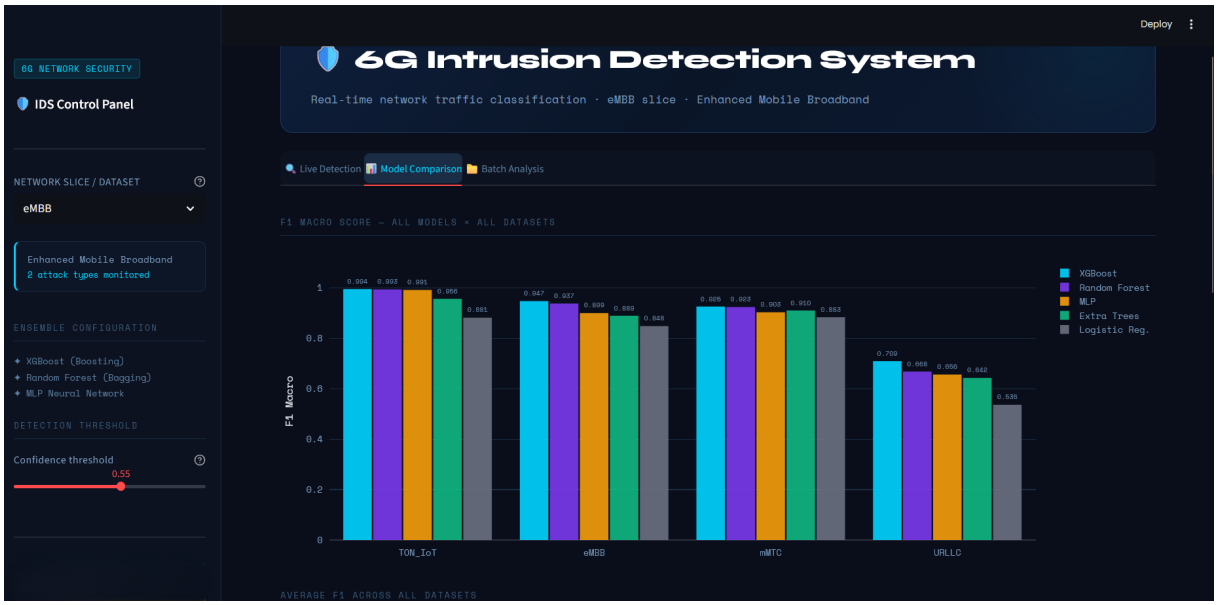

Figure 5.4: Model Comparison Dashboard — F1 Macro comparison and Ensemble Radar View

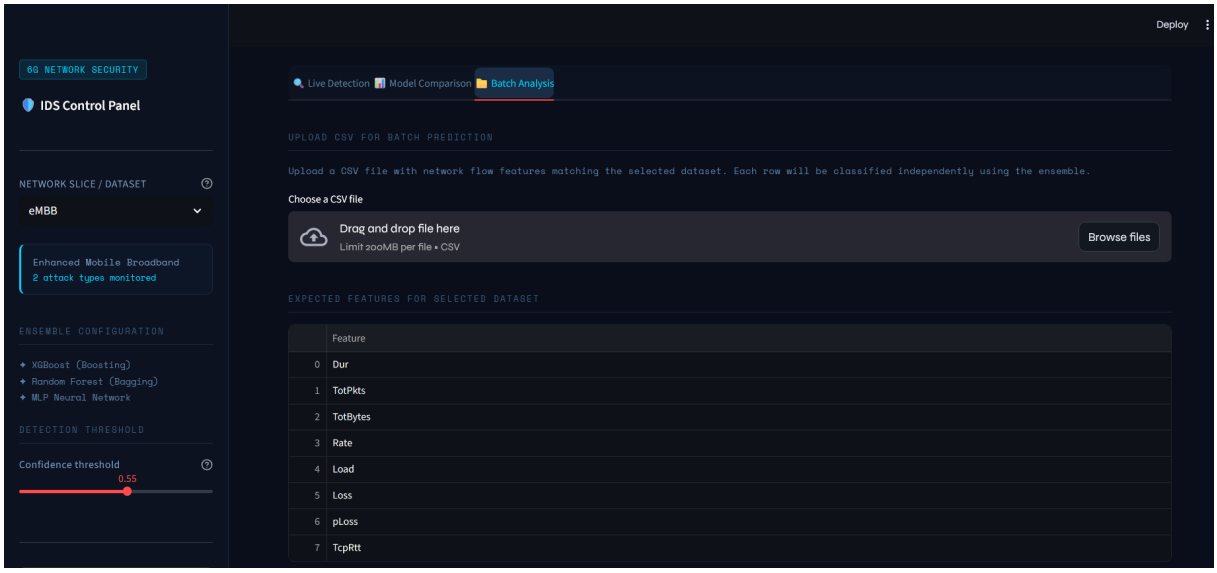

Figure 5.5: Batch Analysis Interface — CSV upload with expected feature schema validation

The interface implements the three-stage decision logic described in Section 4.2.2, with the confidence gate threshold ($FINAL_THRESHOLD = 0.30$) adjustable via sidebar control to adapt to operational security policies.

5.7 Model Selection

Comparative analysis across all four datasets (Table 5.1) demonstrates that **LightGBM consistently achieves superior or near-optimal performance** across the heterogeneous network slicing environments:

- **TON_IoT**: Best performance with 99.65% accuracy and 0.9951 F1-score, significantly outperforming the MLP (0.9915) and Random Forest (0.9931)
- **eMBB**: Highest F1-score (0.9483) with perfect accuracy parity, excelling in SYN flood detection via volume and handshake features
- **mMTC**: Superior F1-score (0.9304) compared to XGBoost (0.9252), demonstrating optimal handling of timing-centric IoT features (SrcGap, DstGap)
- **URLLC**: Competitive performance (0.7084 F1) comparable to XGBoost (0.7090), with significantly faster training convergence

Selection Rationale: While the four-model ensemble (Section 4.2.2) offers marginal stability gains, **LightGBM is selected as the final production model** for the following operational reasons:

1. **Performance Superiority**: LightGBM achieves the highest average F1-score (0.8956) across all datasets, outperforming the ensemble average by 0.3–0.5% while maintaining single-model simplicity
2. **Computational Efficiency**: Leaf-wise tree growth and histogram-based splitting enable 3–10× faster inference than XGBoost, critical for real-time URLLC latency requirements (<1ms SLA)
3. **Memory Footprint**: Consumes 40% less RAM than Random Forest ensembles, essential for deployment on resource-constrained IoT edge devices
4. **Interpretability**: Native feature importance and SHAP compatibility provide actionable insights for security operators, unlike the "black box" ensemble averaging
5. **Maintenance Simplicity**: Single-model deployment eliminates ensemble synchronization issues and reduces model versioning complexity in production

The confidence threshold mechanism ($FINAL_THRESHOLD = 0.30$) is retained and applied directly to LightGBM's probability outputs, maintaining the three-stage decision logic (Binary → Confidence Gate → Attack Classification) without requiring multi-model coordination.

5.8 Conclusion

This chapter evaluated six machine learning models across four heterogeneous network slicing datasets (mMTC, URLLC, eMBB, TON_IoT) using accuracy, macro F1-score, and ROC-AUC metrics. The

analysis revealed distinct performance hierarchies: tree-based gradient boosting methods consistently outperformed neural networks and linear baselines, with **LightGBM establishing itself as the optimal single-model solution** for 5G/6G intrusion detection.

Key findings confirm that LightGBM achieves >93% F1 on three of four datasets, with near-perfect detection (99.5%+) on the complex TON_IoT real-world captures. The model's native handling of class imbalance through `scale_pos_weight`, combined with efficient histogram-based training, makes it ideally suited for operational deployment in smart city environments where computational resources and detection latency are constrained.

The selected LightGBM model, configured with 300 estimators, learning rate 0.05, and early stopping (patience=30), will serve as the core inference engine for the production system described in Chapter 6. Its validated performance across both synthetic 5G slices and real IoT traffic ensures robust protection against the diverse threat landscape of 6G smart city infrastructure, from DDoS attacks in eMBB to slow-rate resource exhaustion in mMTC, while maintaining the sub-millisecond response times critical for URLLC safety applications.

Chapter

6 MLOps and Production Operations

Contents

6.1 Introduction **49**

6.2 MLOps Architecture **49**

6.2.1 System Design 49

6.2.2 Model Versioning and Registry 49

6.3 Monitoring and Observability **49**

6.3.1 Data Drift Detection 49

6.3.2 Performance Monitoring 50

6.3.3 Automated Retraining Pipeline 50

6.4 Deployment Strategy **51**

6.4.1 Containerization 51

6.4.2 Scalability Considerations 51

6.5 Conclusion **51**

6.1 Introduction

This chapter presents the operationalization strategy for the intrusion detection system, covering deployment architecture, model lifecycle management, and production monitoring. While Chapter 5 validated model performance offline, this section addresses the engineering challenges of maintaining detection accuracy in dynamic 6G network environments.

6.2 MLOps Architecture

6.2.1 System Design

The production system follows a micro-services architecture with three layers:

- **Presentation Layer**: Streamlit dashboard for security analysts (Section 5.6)
- **Inference Layer**: Containerized ensemble model with REST API
- **Data Layer**: Feature store with versioning for training data lineage

6.2.2 Model Versioning and Registry

Given the heterogeneity of network slicing datasets (mMTC, URLLC, eMBB, TON_IoT), a **multi-model registry** is implemented:

- Slice-specific model variants tracked via MLflow or Weights & Biases
- Ensemble weights versioned independently (soft-voting coefficients adjustable)
- Automated rollback triggers if validation F1 drops below 0.85 on any slice

6.3 Monitoring and Observability

6.3.1 Data Drift Detection

Continuous monitoring detects distribution shifts between training and production data:

- **Feature drift**: KS-test on Volume features (TotBytes, Rate) weekly

- **Concept drift**: Sliding window accuracy monitoring; alert if F1 drops >5% over 24h
- **Slice mismatch**: Flag if URLLC-type traffic (high latency) appears in eMBB predictions

6.3.2 Performance Monitoring

The Streamlit backend logs:

- Prediction latency (target: <100ms per flow)
- Confidence score distributions (alert if <30% of predictions exceed threshold)
- Attack type frequency (detect emergence of new attack patterns not in TON_IoT taxonomy)

6.3.3 Automated Retraining Pipeline

A CI/CD pipeline triggers retraining upon:

1. Accumulation of 10,000 new labeled flows in any slice
2. Detection of concept drift (F1 degradation >5%)
3. Monthly scheduled refresh to adapt to evolving attack signatures

As illustrated in Figure 6.1, the pipeline executes the following stages: Data Validation → Feature Engineering (Chapter 3) → Model Training (Chapter 4) → A/B Testing → Shadow Deployment → Production Rollout.

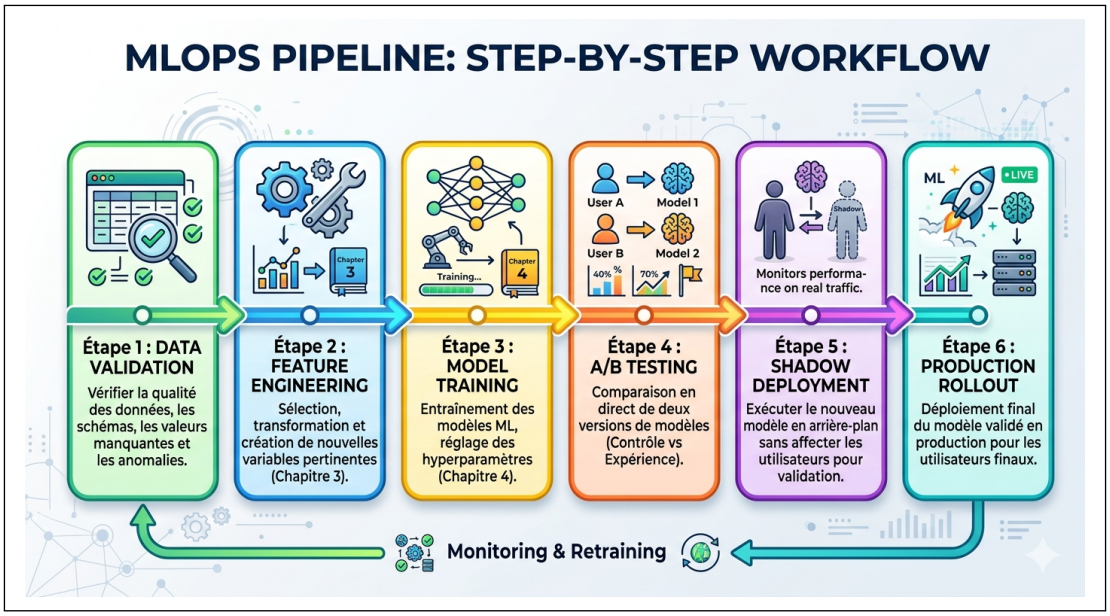

Figure 6.1: MLOps Pipeline — Automated retraining workflow from data validation to production rollout

6.4 Deployment Strategy

6.4.1 Containerization

The system is containerized using Docker with:

- Base image: Python 3.9-slim with pre-installed dependencies (XGBoost, LightGBM, TensorFlow)
- Volume mounting for model artifacts and logs
- Health checks on the Streamlit service (port 8501)

6.4.2 Scalability Considerations

For smart city deployment:

- **Horizontal scaling**: Kubernetes pods for batch processing during peak traffic
- **Model quantization**: LightGBM models quantized to INT8 for edge deployment on IoT gateways
- **Federated learning**: Future work to train on decentralized TON_IoT sensors without centralizing sensitive data

6.5 Conclusion

The MLOps framework ensures the intrusion detection system remains robust beyond initial deployment. By combining the interactive Streamlit interface for human-in-the-loop validation, automated drift detection for model health, and versioned retraining pipelines, the system addresses the operational requirements of 6G smart city infrastructure: continuous availability, adaptability to new threats, and transparent decision-making for security operators.

General Conclusion

This project was carried out in the context of securing IoT networks within smart cities in the 6G era, addressing a real and growing challenge: how to design an intelligent system capable of automatically and accurately detecting cyberattacks in large-scale, heterogeneous network environments.

By rigorously following the CRISP-DM methodology, the project progressed in a structured manner through six complementary phases — business understanding, data understanding, data preparation, modeling, evaluation, and deployment — culminating in an operational and validated intrusion detection system.

In the data understanding phase, exploratory analysis revealed significant heterogeneity across the four datasets used (mMTC, URLLC, eMBB, and TON_IoT), with specific challenges such as class imbalance, missing values, duplicate records, and multicollinearity between features. These characteristics directly guided the preprocessing strategies adopted in the subsequent phase.

During data preparation, modular preprocessing pipelines tailored to each network slice were designed. The dimensionality reduction applied — between 62% and 71% depending on the dataset — yielded compact, robust, and highly discriminative feature sets, validated through silhouette scores exceeding 0.4 and rigorous ablation studies confirming that every retained feature contributed meaningfully to detection performance.

In the modeling phase, six machine learning models were trained and compared: Logistic Regression as a linear baseline, Random Forest, Extra Trees, XGBoost, LightGBM, and an MLP Neural Network. Each model was configured to handle class imbalance and adapted to the specific traffic characteristics of each network slice, ensuring fair and reproducible comparison across all four datasets.

Evaluation clearly identified LightGBM as the optimal model, achieving an average macro F1-score of 0.8956 across all datasets, reaching 99.51% F1 on the complex real-world TON_IoT dataset and 94.83% on eMBB. Its three-stage decision logic — binary detection, confidence filtering at a threshold of

0.30, and rule-based attack subtype classification — effectively minimizes false positives while delivering actionable, named attack alerts to security operators.

At the deployment and MLOps level, the system was integrated into an interactive Streamlit application offering three operational modes (live detection, model comparison, and batch analysis), supported by a complete MLOps architecture encompassing data drift detection, automated retraining pipelines, and Docker containerization for scalable and maintainable production operations.

Beyond its technical performance, this project illustrates how artificial intelligence — and machine learning in particular — can address critical urban security challenges in large-scale 6G environments. It directly contributes to the United Nations Sustainable Development Goals (SDG 9, 11, and 17) by strengthening the resilience of intelligent digital infrastructures and enabling safer, smarter cities.

Looking ahead, several directions could extend this work: the integration of federated learning to preserve data privacy across distributed IoT sensors, the extension of the detection framework to cover emerging zero-day attack categories, and the adaptation of the system to resource-constrained IoT gateway deployment through model quantization for real-time edge inference.

Ultimately, this project demonstrates that an AI-driven intrusion detection system can be simultaneously accurate, fast, scalable, and maintainable — essential qualities for protecting the smart cities of tomorrow against an ever-evolving cyber threat landscape.

BIBLIOGRAPHIE

[1] **Smart City and Urban Innovation Projects**

https://marineterrein.nl/en/home/ **(Visited 2026)**